SR UNIVERSITY

AI Assistant Coding
Assignment-17.3

Name of Student : V. Abhinav Batch : 41
Enrollment No. : 2303A52486

Task-1: Employee Dataset Cleaning

Use AI assistance to clean an employee information dataset.

Prompt:

Clean an employee dataset by handling missing values in salary, designation, and experience while removing duplicate employee IDs. Standardize job titles to lowercase and encode categorical features such as designation and employment type for analysis readiness.

Before Pre Processing:

```
df = pd.read_csv("HRDataset_v14.csv")
df.head()
```

	Employee_Name	EmpID	MarriedID	MaritalStatusID	GenderID	EmpStatusID	DeptID	PerfScoreID	FromDiversityJobFairID	Salary	...	ManagerName	ManagerID	RecruitmentSour
0	Adinolfi, Wilson K	10026	0	0	1	1	5	4	0	62506	...	Michael Albert	22.0	Linke
1	Ait Sidi, Karthikeyan	10084	1	1	1	5	3	3	0	104437	...	Simon Roup	4.0	Inde
2	Akinkuolie, Sarah	10196	1	1	0	5	5	3	0	64955	...	Kissy Sullivan	20.0	Linke
3	Alagbe,Trina	10088	1	1	0	1	5	3	0	64991	...	Elijiah Gray	16.0	Inde
4	Anderson, Carol	10069	0	2	0	5	5	3	0	50825	...	Webster Butler	39.0	Google Sear

5 rows × 36 columns

Code:

```python
import pandas as pd
from sklearn.preprocessing import LabelEncoder

# Load dataset
df = pd.read_csv("HRDataset_v14.csv")

# Handle missing values
df['Salary'] = df['Salary'].fillna(df['Salary'].median())
df['Position'] = df['Position'].fillna("unknown")

# Remove duplicates based on Employee ID
df = df.drop_duplicates(subset='EmpID')
```

```
# Standardize text (lowercase)
df['Position'] = df['Position'].str.lower()

# Encode categorical features
le = LabelEncoder()
df['Position_encoded'] = le.fit_transform(df['Position'])
df['EmploymentStatus_encoded'] =
le.fit_transform(df['EmploymentStatus'])

# Save cleaned dataset
df.to_csv("clean_employee_data.csv", index=False)

df.head()
```

Output:

	Employee_Name	EmpID	MarriedID	MaritalStatusID	GenderID	EmpStatusID	DeptID	PerfScoreID	FromDiversityJobFairID	Salary	...	RecruitmentSource	PerformanceScore
0	Adinolfi, Wilson K	10026	0	0	1	1	5	4	0	62506	...	LinkedIn	Exceeds
1	Ait Sidi, Karthikeyan	10084	1	1	1	5	3	3	0	104437	...	Indeed	Fully Meets
2	Akinkuolie, Sarah	10196	1	1	0	5	5	3	0	64955	...	LinkedIn	Fully Meets
3	Alagbe,Trina	10088	1	1	0	1	5	3	0	64991	...	Indeed	Fully Meets
4	Anderson, Carol	10069	0	2	0	5	5	3	0	50825	...	Google Search	Fully Meets

5 rows × 38 columns

Explanation:

This task focuses on preparing employee data by handling missing values in important fields like salary, designation, and experience to ensure data completeness. Duplicate employee IDs are removed to maintain data integrity, and job titles are standardized for consistency. Finally, categorical features are encoded so the dataset becomes suitable for HR analytics and machine learning models.

Task 2: Student Performance Data Preprocessing

Preprocess student performance data using AI assistance.

Prompt:

Preprocess student performance data by converting exam dates into datetime format and extracting features such as month, semester, and day. Normalize total marks and detect outliers using statistical methods like IQR or Z-score. Prepare the dataset for academic performance analysis and modeling.

Before Preprocessing:

	gender	race/ethnicity	parental level of education	lunch	test preparation course	math score	reading score	writing score
0	female	group B	bachelor's degree	standard	none	72	72	74
1	female	group C	some college	standard	completed	69	90	88
2	female	group B	master's degree	standard	none	90	95	93
3	male	group A	associate's degree	free/reduced	none	47	57	44
4	male	group C	some college	standard	none	76	78	75

Code:

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler

# Load dataset
df = pd.read_csv("StudentsPerformance.csv")

# Create fake exam_date (if not present)
df['exam_date'] = pd.date_range(start='2023-01-01', periods=len(df),
freq='D')

# Convert to datetime
df['exam_date'] = pd.to_datetime(df['exam_date'])

# Extract features
df['month'] = df['exam_date'].dt.month
df['day'] = df['exam_date'].dt.day
df['semester'] = df['month'].apply(lambda x: 1 if x <= 6 else 2)

# Total marks
```

```
df['total_marks'] = df[['math score', 'reading score', 'writing
score']].sum(axis=1)

# Normalize marks
scaler = MinMaxScaler()
df['total_marks_normalized'] =
scaler.fit_transform(df[['total_marks']])

# Outlier removal using IQR
Q1 = df['total_marks'].quantile(0.25)
Q3 = df['total_marks'].quantile(0.75)
IQR = Q3 - Q1

df = df[(df['total_marks'] >= Q1 - 1.5 * IQR) &
        (df['total_marks'] <= Q3 + 1.5 * IQR)]

df.to_csv("processed_student_data.csv", index=False)

df.head()
```

Output:

...

	gender	race/ethnicity	parental level of education	lunch	test preparation course	math score	reading score	writing score	exam_date	month	day	semester	total_marks	total_marks_normalized
0	female	group B	bachelor's degree	standard	none	72	72	74	2023-01-01	1	1	1	218	0.699634
1	female	group C	some college	standard	completed	69	90	88	2023-01-02	1	2	1	247	0.805861
2	female	group B	master's degree	standard	none	90	95	93	2023-01-03	1	3	1	278	0.919414
3	male	group A	associate's degree	free/reduced	none	47	57	44	2023-01-04	1	4	1	148	0.443223
4	male	group C	some college	standard	none	76	78	75	2023-01-05	1	5	1	229	0.739927

Explanation:

This task involves transforming raw student performance data into a structured format by converting exam dates into datetime and extracting useful features like month, day, and semester. It also includes normalizing total marks to bring values into a consistent range and detecting outliers using statistical methods. These steps improve data quality and make it ready for analysis and predictive modeling.

SR UNIVERSITY

Task 3: Student Health Data Preparation

Clean and preprocess student health records using AI scripts.

Prompt:

Clean student health records by handling missing values in height, weight, and BMI. Encode categorical variables such as medical condition and fitness status, then scale numerical features using standard scaling. Finally, split the dataset into training and testing sets for predictive analysis.

Before Preprocessing:

	Timestamp	Name	Age	Gender	Height(in CMs/ Inches)	Weight(in KGs)	Do You Wear Spectacles?	Food Preference	Have you visited CU hospital recently?	Purpose of visiting hospital	Were you referred to Another Hospital ?	Has CU hospital helped you?	Have you received medical leave from cu hospital?	How many hours do you sleep in a day?	Do you feel mentally exhausted sometimes?	Rate your mental health on a scale of 1 to 5	Do you need Mental Relaxation Sessions
0	5/8/2023 11:58	Raj	20	Male	176.0	67.0	Yes	Non-VEG	yes	Body Pain, Fever	No	yes	yes	7 hours	Yes	2	yes
1	5/8/2023 11:59	Jatin	20	Male	170.0	68.0	No	VEG	yes	Fever	No	yes	No	7 hours	Yes	1	yes
2	5/8/2023 11:59	Aplavya Kumar	18	Male	185.0	65.0	Yes	VEG	yes	Fever	No	yes	No	8 hours	Yes	3	no
3	5/8/2023 12:03	Nikhil sharma	20	Male	170.0	72.0	No	Non-VEG	No	NOT VISITED	Yes	No	No	less than 6 hours	Yes	3	yes
4	5/8/2023 12:09	Sachin	20	Male	181.0	81.0	Yes	Non-VEG	yes	Injuries	No	yes	yes	less than 6 hours	No	2	no

Code:

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split

# Load dataset
df = pd.read_csv("/content/Student Health.csv")
# Clean column names: strip whitespace and convert to lowercase
df.columns = df.columns.str.strip().str.lower()

# Rename specific columns after general cleaning
# 'bmi', 'medical_condition', 'fitness_status' should already be
correctly named
# after lowercasing and stripping
df = df.rename(columns={
    'height(in cms/ inches)': 'height',
    'weight(in kgs)': 'weight',
    'purpose of visiting hospital': 'medical_condition',
```

```
    'do you need mental relaxation sessions': 'fitness_status'
}, errors='ignore') # Use errors='ignore' to prevent error if a key is
not found

# Handle missing values
df['height'] = df['height'].fillna(df['height'].mean())
df['weight'] = df['weight'].fillna(df['weight'].mean())
df['bmi'] = df['bmi'].fillna(df['bmi'].mean())

# Encode categorical features
le = LabelEncoder()
df['medical_condition_encoded'] =
le.fit_transform(df['medical_condition'])
df['fitness_status_encoded'] = le.fit_transform(df['fitness_status'])

# Scale numerical features
scaler = StandardScaler()
df[['height', 'weight', 'bmi']] = scaler.fit_transform(df[['height',
'weight', 'bmi']])

# Split dataset
X = df.drop(columns=['fitness_status', 'medical_condition'])
y = df['fitness_status_encoded']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print(X_train.head())
print(y_train.head())
print("Train shape:", X_train.shape)
print("Test shape:", X_test.shape)
```

Output:

```
...
              timestamp            name  age  gender    height    weight  \
10   5/8/2023 12:15           Karan   22    Male -0.023435 -0.424663
130  5/8/2023 15:00       Vishakha    23  Female -0.079878  0.577844
51   5/8/2023 13:03  Arihant Kumar    18    Male  0.258785  0.511010
11   5/8/2023 12:16           Tanya   19  Female -0.813649 -0.758831
114  5/8/2023 14:44            Tany   21  Female -0.192766 -1.360335

     do you wear  spectacles? food preference  \
10                      No            VEG
130                    Yes            VEG
51                     Yes        Non-VEG
11                      No            VEG
114                     No            VEG
```

```
    have you visited cu hospital recently?  \
10                                     yes
130                                    yes
51                                     yes
11                                     yes
114                                    yes

    were you referred to another hospital ? has cu hospital helped you?  \
10                                     No                        yes
130                                   Yes                        yes
51                                     No                         No
11                                     No                        yes
114                                   Yes                        yes

    have you received medical leave from cu hospital?  \
10                                                yes
130                                               yes
51                                                 No
```

Explanation:

This task aims to clean student health records by filling missing values in key attributes such as height, weight, and BMI. It encodes categorical variables like medical condition and fitness status into numerical form and scales numerical features for uniformity. Finally, the dataset is split into training and testing sets to support predictive health analysis.

Task 4: Real-Time Application: Ride-Sharing Data Cleaning

Prompt:

Process ride-sharing trip data by standardizing location names and handling missing fare values using median imputation. Remove duplicate entries, normalize driver ratings, and generate a summary report of improvements. Prepare the dataset for operational analytics and decision-making.

Before Preprocessing:

```
'''
```

	Unnamed: 0	key	fare_amount	pickup_datetime	pickup_longitude	pickup_latitude	dropoff_longitude	dropoff_latitude	passenger_count
0	24238194	2015-05-07 19:52:06.0000003	7.5	2015-05-07 19:52:06 UTC	-73.999817	40.738354	-73.999512	40.723217	1.0
1	27835199	2009-07-17 20:04:56.0000002	7.7	2009-07-17 20:04:56 UTC	-73.994355	40.728225	-73.994710	40.750325	1.0
2	44984355	2009-08-24 21:45:00.00000061	12.9	2009-08-24 21:45:00 UTC	-74.005043	40.740770	-73.962565	40.772647	1.0
3	25894730	2009-06-26 08:22:21.0000001	5.3	2009-06-26 08:22:21 UTC	-73.976124	40.790844	-73.965316	40.803349	3.0
4	17610152	2014-08-28 17:47:00.000000188	16.0	2014-08-28 17:47:00 UTC	-73.925023	40.744085	-73.973082	40.761247	5.0

Code:

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler

# Load dataset
df = pd.read_csv("uber.csv")

# Standardize locations (lowercase)
df['pickup_datetime'] = df['pickup_datetime'].str.lower()

# Fill missing fare
df['fare_amount'] =
df['fare_amount'].fillna(df['fare_amount'].median())

# Remove duplicates
df = df.drop_duplicates()

# Normalize driver rating (create dummy if not present)
if 'driver_rating' not in df.columns:
    df['driver_rating'] = 4.0  # dummy constant

scaler = MinMaxScaler()
df['driver_rating_normalized'] =
scaler.fit_transform(df[['driver_rating']])

# Summary
summary = {
    "Initial rows": len(df),
    "Missing values handled": df.isnull().sum().sum(),
    "Final rows": len(df)
}

print("Data Cleaning Summary:", summary)

df.to_csv("clean_rides_data.csv", index=False)
df.head()
```

Output:

Data Cleaning Summary: {'Initial rows': 98305, 'Missing values handled': np.int64(8), 'Final rows': 98305}

	Unnamed: 0	key	fare_amount	pickup_datetime	pickup_longitude	pickup_latitude	dropoff_longitude	dropoff_latitude	passenger_count	driver_rating
0	24238194	2015-05-07 19:52:06.0000003	7.5	2015-05-07 19:52:06 utc	-73.999817	40.738354	-73.999512	40.723217	1.0	4.0
1	27835199	2009-07-17 20:04:56.0000002	7.7	2009-07-17 20:04:56 utc	-73.994355	40.728225	-73.994710	40.750325	1.0	4.0
2	44984355	2009-08-24 21:45:00.00000061	12.9	2009-08-24 21:45:00 utc	-74.005043	40.740770	-73.962565	40.772647	1.0	4.0
3	25894730	2009-06-26 08:22:21.0000001	5.3	2009-06-26 08:22:21 utc	-73.976124	40.790844	-73.965316	40.803349	3.0	4.0
4	17610152	2014-08-28 17:47:00.000000188	16.0	2014-08-28 17:47:00 utc	-73.925023	40.744085	-73.973082	40.761247	5.0	4.0

Explanation:

This task focuses on cleaning ride-sharing data by standardizing location information and handling missing fare values using median imputation. Duplicate records are removed, and driver ratings are normalized to ensure consistency across the dataset. A summary of data cleaning improvements is generated, making the dataset ready for operational insights and analytics.